Design Document - Assignment 2, PS04: Hex Game AI using Monte Carlo Tree Search

BITS Pilani WILP - MTech (AI & ML) - S2_2025-2026 - AIMLCZG557 / AECLZG557 - Group ID: G116

Source file: G116_A2_PS04.py | Input: inputPS04.txt | Output: outputPS04.txt

1. Problem Formulation and Game Model

Hex is a deterministic, perfect-information, zero-sum, two-player game with no draws. On an N x N board (7 <= N <= 11) Player A (symbol 1, the AI agent, moves first) must connect the TOP row to the BOTTOM row and Player B (symbol 2, the human) must connect the LEFT column to the RIGHT column. The hexagonal topology is encoded on a square matrix by giving every cell (r,c) exactly six neighbours:

```
Orthogonal : (r-1,c) (r+1,c) (r,c-1) (r,c+1)
Diagonal   : (r-1,c+1) Top-Right, (r+1,c-1) Bottom-Left
Forbidden  : (r-1,c-1) Top-Left , (r+1,c+1) Bottom-Right
```

State: the N x N matrix plus the side to move. Action: place one stone on any empty cell (stones are never moved or removed). Terminal test: a chain of own stones joins the two own edges; because a filled Hex board always contains exactly one winner, the game can never end in a draw. Utility: +1 win / 0 loss. Since the player to move is determined by the stone counts (A moves first), the turn is derived from the board itself and is never stored separately - this makes any legal position resumable from the input file.

2. Solution Architecture (modules of G116_A2_PS04.py)

Component	Data structure	Responsibility
HexBoard	flat list of N*N ints + pre-computed neighbour table (tuple of tuples)	Board state, legal moves, insert/delete of stones with validation, connectivity test (iterative DFS over a bytearray), shortest winning path (BFS), rendering.
MCTSNode	n-ary tree: children list, untried-move list, (visits, wins) counters	Stores one position; implements the UCT selection formula, the expansion of one untried child and the backpropagation update.
MCTSAgent	search tree + random.Random	Runs the four MCTS phases until the time budget expires and returns the move, win probability, rollout count, nodes expanded, tree depth and elapsed time.
HexGame	statistics lists	Alternating turn loop, human-move validation and re-prompting, per-turn logging, end-of-game report.
OutputLogger	file handle	Mirrors every line to the console and to outputPS04.txt.

3. The MCTS Agent (UCT)

Each call to choose_move grows an asymmetric search tree by repeating the four canonical phases until the time limit read from inputPS04.txt is reached (anytime property):

```
while time.perf_counter() < deadline:
  node, state <- root, board.copy()
  # 1 SELECTION      : descend fully expanded nodes with the UCT rule
  while node.fully_expanded() and node.children:
      node <- argmax_child ( w_i/n_i + c * sqrt( ln(n_parent)/n_i ) )
      state.place(node.move); switch side to move
  # 2 EXPANSION      : attach ONE random untried child
  m <- node.untried_moves.pop(); state.place(m); node <- node.add(m)
  # 3 SIMULATION     : uniformly random play-out to a terminal state
  winner <- rollout(state)
  # 4 BACKPROPAGATION: update (visits, wins) up to the root
  while node: node.visits += 1; node.wins += (winner == node.owner)
return most-visited child of the root      # robust-child criterion
```

Exploration constant c = sqrt(2), the theoretical UCB1 value for rewards in [0,1], which is exactly the win/loss reward used here. Wins are counted from the point of view of the player who created the node, so sibling UCT values are directly comparable. The final answer is the most visited child (robust child) rather than the highest win rate, because the visit count is far less noisy for rarely sampled moves; its win rate is reported as the heuristic score of the move.

3.1 Rollout optimisation (the hot path)

A naive play-out places one random stone at a time and tests the terminal condition after every ply: O(N^2) placements x O(N^2) test = O(N^4). We exploit the no-draw theorem instead: the empty cells are shuffled once and dealt alternately to both players, and the single resulting full board is tested only for A's top-bottom connection (if A is not connected, B necessarily is). This is statistically identical to move-by-move random play but costs one shuffle plus one flood fill, i.e. O(N^2) per rollout. Measured effect on a 7x7 board with a 2000 ms budget: about 45,000 - 51,000 rollouts per move (~23,000 rollouts/s) instead of a few thousand.

3.2 Domain knowledge injected at the root

Pure UCT is tactically blind in the last plies because a single winning cell is diluted among ~N^2 root moves. Two O(N^2 . |legal|) scans are therefore run before the search: (a) if the agent has an immediate winning move it is played at once (score +INF); (b) if the opponent has a winning reply, the root move list is restricted to the blocking cells and MCTS searches only those. The scans cost under 1% of the budget and remove the only class of blunder observed with plain UCT.

4. Complexity Analysis

Operation	Complexity	Remark
neighbours(r,c)	O(1)	pre-computed table, shared by every board copy
legal_moves / board copy	O(N^2)	single pass over the flat list
is_connected / winner / winning_path	O(N^2)	each cell is pushed once, 6 edges per cell -> O(6N^2) = O(N^2)
one rollout (simulation)	O(N^2)	shuffle + fill + one flood fill
one MCTS iteration	O(d.b + N^2)	d = tree depth reached, b = branching factor (b <= N^2); the UCT scan is O(b) per level
one move with S iterations	O(S.(d.b + N^2))	S is fixed by the time limit T, so the wall-clock cost is O(T) by construction
complete game	O(N^2) moves x O(T)	at most N^2 plies
memory	O(S) nodes, O(N^2) per open node	~50,000 nodes per move on 7x7; the tree is discarded after each move

The size of the state space is bounded by 3^(N^2) (1.7 x 10^57 for N=11) and the game tree by (N^2)! move orderings, so exhaustive search is impossible. MCTS never enumerates this space: its cost depends only on the time budget, and the statistical error of a node's win estimate falls as O(1/sqrt(n)). UCT is asymptotically consistent - as S -> infinity the visit distribution converges to the minimax-optimal move.

5. Alternate Way of Modelling the Problem and its Performance Implications

Alternative: model Hex as a classical adversarial search problem and solve it with depth-limited Minimax + alpha-beta pruning over an electrical-resistance / two-distance evaluation function. Each player's position is viewed as a resistor network (own stone = 0 ohm, empty = 1 ohm, opponent stone = infinite ohm) and the score is log(R_B / R_A); alternatively the 'two-distance' shortest-connection metric is computed with a Dijkstra-like relaxation. Incremental connectivity is maintained with a union-find forest holding four virtual edge nodes (TOP, BOTTOM, LEFT, RIGHT), which makes the terminal test O(alpha(N^2)) instead of O(N^2).

Performance implications

- Cost: with branching factor b = N^2 (49 to 121) alpha-beta explores O(b^(d/2)) nodes under perfect move ordering. For N=11 and depth 6 that is ~1.7 x 10^6 evaluations, and each evaluation is itself O(N^2 log N) - roughly 10^8 operations, far beyond 2 s in CPython, whereas MCTS delivers a usable answer for any budget.
- Not anytime: alpha-beta must finish a whole ply (or use iterative deepening and throw work away) before its answer is trustworthy; MCTS can be interrupted at any instant.
- Knowledge dependence: its strength is entirely determined by a hand-crafted evaluation function. Hex has no material and no simple positional features, which is exactly why hand-written evaluators plateaued and why every strong modern Hex engine (MoHex, MoHex-3HNN) is MCTS-based.
- Where it wins: it is exact and deterministic near the end of the game, it proves wins instead of estimating them, and it reuses transposition tables well. A practical hybrid - alpha-beta for the last plies, MCTS elsewhere - is what our root tactical filter approximates cheaply.
- Union-find variant of the board: placement becomes near O(1) and the terminal test O(1), but MCTS rollouts need cheap undo, and a roll-backable DSU (no path compression) plus its per-node Python object overhead measured slower than the plain bytearray flood fill for N <= 11; it only pays off for N >= 19.

6. What if the Board is Larger than 11 x 11?

Yes - the program still terminates in a deterministic amount of time, because MCTS is an anytime algorithm bounded by the time limit, not by the size of the tree: every move returns within T milliseconds for any N. What degrades is not the running time but the decision quality. Rollout cost grows as O(N^2) while the number of root moves also grows as N^2, so the number of samples per root move collapses quadratically: on 19x19 the same 2 s yields roughly 7x fewer rollouts spread over 7x more candidates (~50x fewer samples per move), and the agent degenerates towards random play. Suitable algorithms for such large state spaces:

- RAVE / AMAF (Rapid Action Value Estimation): share the outcome of a rollout with every move played in it, so one simulation updates many siblings - the standard fix for a huge branching factor and the basis of MoHex.
- Heavy (informed) play-outs and pattern policies: bridge-completion and local-reply patterns make each sample far more informative than a uniform random one.
- Progressive widening / progressive bias: expand only the k most promising children of a node, with k growing as sqrt(n), which caps the effective branching factor.
- PUCT guided by a deep policy/value network (AlphaZero, MoHex-3HNN): the value head removes the need to roll out to the end, and the policy head prunes the branching factor from N^2 to a handful.
- Hex-specific inference: dead-cell, captured-cell and inferior-cell pruning plus H-search virtual connections provably remove moves from consideration and shrink the tree before any sampling.
- Parallelisation: root/tree parallel MCTS with virtual loss, or transposition-aware DAG-MCTS, to multiply the number of samples obtainable inside the same wall-clock budget.

7. Game Theory in Current AI Trends (paper review)

Paper: Meta FAIR Diplomacy Team, 'Human-level play in the game of Diplomacy by combining language models with strategic reasoning' (CICERO), Science 378(6624), 2022. Diplomacy is a seven-player, simultaneous-move, non-zero-sum negotiation game, so - unlike Hex - it cannot be solved by pure adversarial search: an agent must both predict human behaviour and negotiate in free-form natural language. CICERO couples an LLM-based dialogue module with a planning module that runs piKL, a regret-minimisation (equilibrium-search) procedure which iteratively refines a policy towards a Nash equilibrium while a KL penalty keeps it close to an 'anchor' policy learned by imitation from human games. The equilibrium therefore stays human-compatible instead of converging to an unexploitable but alien strategy, and the negotiated joint plan is then verbalised by the language model and filtered for consistency. CICERO reached more than twice the average human score in 40 anonymous online games.

Relevance to this assignment and to the wider trend: (i) the architecture is exactly ours in spirit - a learned or uniform prior combined with search-time refinement; our MCTS refines a uniform prior with rollouts, CICERO refines an LLM prior with regret minimisation, and AlphaZero refines a neural prior with PUCT. (ii) The same game-theoretic template now underpins several AI trends: RLHF/DPO alignment is a Stackelberg game between a policy and a reward model, self-play and debate are used to elicit truthful LLM behaviour, and adversarial (minimax) training - the GAN objective of Goodfellow et al., 2014 - drives both neural fake news generation and its detection (e.g. Grover, Zellers et al., 2019), where generator and detector form a two-player zero-sum game whose equilibrium determines whether synthetic text remains detectable. (iii) It confirms the central lesson of our experiments: search on top of a prior beats either component alone.

8. Design Trade-offs

Trade-off	Decision taken and why
Exploration constant c	c = sqrt(2). Smaller c exploits, producing a deep narrow tree that can miss a better sibling; larger c explores, producing a wide shallow tree with noisy estimates. sqrt(2) is the UCB1 optimum for rewards in [0,1] and behaved best in our runs.
Time budget vs quality	Rollouts grow linearly with T while the estimation error falls only as 1/sqrt(S): doubling the budget cuts the error by ~30%. Diminishing returns justify the 2000 ms default.
Light vs heavy play-outs	Uniform random (light) play-outs were kept: they are ~10x faster, and on a 7x7-11x11 board the extra samples outweigh the per-sample accuracy of pattern-guided play-outs.
Full-board shuffle vs step-by-step play-out	The no-draw theorem lets us fill the board in one pass, turning an O(N^4) play-out into O(N^2). Cost: the play-out cannot stop early when a win appears, but a single flood fill is still much cheaper.
Robust child vs max win-rate	The most visited child is returned; the maximum win rate is often an artefact of 2-3 lucky samples on a rarely visited move.
Pure UCT vs tactical root filter	A small amount of domain knowledge (win/block detection) was added; it costs <1% of the budget and eliminates end-game blunders, at the price of no longer being 'knowledge free'.
Flat list vs list of lists / bitboards	A flat list with a pre-computed neighbour table gives O(1) adjacency and the cheapest possible copy. Bitboards would be faster still but unreadable, and the report values documented, modular code.
Tree reuse between moves	Not implemented: each move starts from a fresh root. Reuse would save ~10-20% of the samples but keeps a large sub-tree alive and complicates the code; memory already reaches ~50,000 nodes/move.

9. Testing, Error Handling and Edge Cases

Seven test cases are built into the program and are executed with 'python G116_A2_PS04.py --test'; their output is appended to outputPS04.txt. All seven pass.

Case	What is verified	Expected / observed result
TC1	Adjacency rules: six neighbours, forbidden Top-Left and Bottom-Right diagonals, board corners	neighbours(3,3) = the six legal cells; (2,2) and (4,4) absent; (0,0) has only two neighbours - PASSED
TC2	Terminal detection and winning path on the 5x5 example of the problem statement	Winner = A, path (0,2)->(1,2)->(1,3)->(2,3)->(3,3)->(4,2) using the Bottom-Left diagonal - PASSED
TC3	Player B left-to-right connection	Winner = B - PASSED
TC4	The agent plays an immediate winning move	Move (6,2) with score +INF - PASSED
TC5	Legality, statistics and respect of the time budget	legal move, 7,995 rollouts in 500 ms - PASSED
TC6	Malformed human input: '', 'abc', '3', '1,2,3', 'x,y', out-of-range, occupied cell	every case rejected with a specific message; '(4, 1)' accepted - PASSED
TC7	Data-structure and file edge cases: full board, duplicate insert, delete from an empty cell, N outside 7..11, missing input file	BoardFullError / InvalidMoveError / InvalidBoardError raised with descriptive messages - PASSED

Run-time robustness: the human is re-prompted after every invalid move (at most 10 attempts, then the game ends cleanly); EOF or Ctrl+C stops the program with a message and a final report instead of a traceback; a missing or malformed inputPS04.txt falls back to an interactive setup; an inconsistent board (stone counts that cannot arise with A moving first) is rejected; a board that is already terminal is reported immediately.

10. Input / Output Format and Observed Results

inputPS04.txt - line 1: board size N (7..11); line 2: time limit per AI move in ms; following lines: the N x N matrix of 0/1/2 (blank lines and '#' comments are ignored, an absent matrix means an empty board). Every turn is echoed to the console and to outputPS04.txt with the move, the search statistics and the current board; the transcript ends with the GAME OVER summary.

```
====================Turn 13==============================
Player                    : A (AI)
Move Selected             : (6,2)
Search Algorithm          : MCTS(UCT) + immediate-win detection
Simulations (Rollouts)    : 0        Nodes Expanded : 34
Heuristic Score (Win Prob): +INF     Execution Time : 0 ms
Terminal State            : YES      Winner         : Player A
Winning Path : (0,4)->(1,3)->(2,3)->(3,3)->(4,3)->(5,3)->(6,2)
```

Recorded game (7x7, 2000 ms/move, transcript in outputPS04.txt): Player A (AI) won in 13 turns - 7 AI moves and 6 human moves; average 39,453 rollouts and 39,455 nodes expanded per move; average tree depth 3.6 (maximum 4); average AI move time 1,714 ms; maximum 47,629 nodes in a single move. The reported win probability rose monotonically from 0.598 on the opening move to 0.942 and then to +INF, which is the expected behaviour of a converging UCT estimate and a good sanity check on the implementation.